Exercise 1: Hash Map

2. Product Class

```
class Product {
  int productId;
  String productName;
  int quantity;
  double price;
  Product(int id, String name, int qty, double price) {
     this.productId = id;
     this.productName = name;
     this.quantity = qty;
     this.price = price;
  }
  public String toString() {
     return productId + " " + productName + " Qty:" + quantity + " Price:" +
price;
  }
}
```

---

3. **Inventory System**

```
import java.util.*;
public class InventorySystem {
  public static void main(String args[]) {
     HashMap<Integer, Product> inventory = new HashMap<>();
     inventory.put(101,new Product(101,"Laptop",10,55000));
     inventory.put(102,new Product(102,"Mouse",30,500));
     inventory.put(102,new Product(102,"Wireless Mouse",25,700));
     inventory.remove(101);
     for(Product p: inventory.values())
```

```
        System.out.println(p);
    }
}
```

Output

102 Wireless Mouse Qty:25 Price:700.0

Exercise 2:

Product Class

```
class Product{
   int id;
   String name;

   Product(int id,String name){
      this.id=id;
      this.name=name;
   }
}
```

Linear search

```
public class LinearSearchDemo{

    public static void main(String args[]){

        Product p[]={
              new Product(1,"Laptop"),
              new Product(2,"Mouse"),
              new Product(3,"Keyboard")
        };

        String key="Mouse";

        boolean found=false;
```

```
        for(Product x:p){

            if(x.name.equals(key)){
                System.out.println("Product Found : "+x.name);
                found=true;
            }
        }

        if(!found)
            System.out.println("Not Found");
    }
}
```

0/p: Product Found : Mouse

Binary Search

```
import java.util.*;

public class BinarySearchDemo{

   public static void main(String args[]){

      String products[]={"Keyboard","Laptop","Mouse"};

      Arrays.sort(products);

      int index=Arrays.binarySearch(products,"Mouse");

      if(index>=0)
         System.out.println("Found at index "+index);
      else
         System.out.println("Not Found");
```

```
  }
}
```

Output

Found at index 2

Exercise 3:

Order Class

```
class Order{

  int orderId;
  String customerName;
  double totalPrice;

  Order(int id,String name,double price){

     orderId=id;
     customerName=name;
     totalPrice=price;
  }
}
```

Bubble Sort

```
public class BubbleSortDemo{

  public static void main(String args[]){

     double price[]={4500,1200,8900,3000};
```

```
    for(int i=0;i<price.length-1;i++){

        for(int j=0;j<price.length-i-1;j++){

            if(price[j]>price[j+1]){

                double temp=price[j];
                price[j]=price[j+1];
                price[j+1]=temp;
            }
        }
    }

    for(double x:price)
        System.out.print(x+" ");
  }
}
```

Output

1200.0 3000.0 4500.0 8900.0

Quick Sort

```
import java.util.*;

public class QuickSortDemo{

  public static void main(String args[]){

    int arr[]={4500,1200,8900,3000};
```

```
    Arrays.sort(arr);

    System.out.println(Arrays.toString(arr));
  }
}
```

Output

[1200, 3000, 4500, 8900]

Exercise 4:

Employee Class

```
class Employee{

  int id;
  String name;

  Employee(int id,String name){

    this.id=id;
    this.name=name;
  }
}
```

Program

```
public class EmployeeDemo{

  public static void main(String args[]){
```

```
        Employee emp[]=new Employee[3];

        emp[0]=new Employee(1,"John");
        emp[1]=new Employee(2,"David");
        emp[2]=new Employee(3,"Mary");

        for(Employee e:emp)
           System.out.println(e.id+" "+e.name);

        emp[1]=null;

        System.out.println("After Deletion");

        for(Employee e:emp){

           if(e!=null)
           System.out.println(e.id+" "+e.name);
        }
    }
}
```

Output

```
1 John
2 David
3 Mary

After Deletion

1 John
3 Mary
```

Exercise 5:

Exercise 5: Task Management System

Linked Lists

Singly Linked List

Each node contains:

- Data
- Next Pointer

Doubly Linked List

Each node contains:

- Previous Pointer
- Data
- Next Pointer

Program

```
import java.util.*;

public class TaskDemo{

  public static void main(String args[]){

    LinkedList<String> tasks=new LinkedList<>();

    tasks.add("Coding");
```

```
        tasks.add("Testing");
        tasks.add("Documentation");

        System.out.println(tasks);

        tasks.remove("Testing");

        System.out.println(tasks);
    }
}
```

Output

[Coding, Testing, Documentation]

[Coding, Documentation]

Analysis

Operation	Complexity
Add	O(1)
Search	O(n)
Traverse	O(n)
Delete	O(n)

Advantages

- Dynamic size
- Easy insertion

- Easy deletion

Exercise 6: Library Management System

Linear Search

```
public class LibrarySearch{

  public static void main(String args[]){

     String books[]={"Java","Python","C++","AI"};

     String key="Python";

     for(String b:books){

        if(b.equals(key))
           System.out.println("Book Found");
     }
  }
}
```

Output

Book Found

Binary Search

```
import java.util.*;

public class BinaryLibrary{
```

```
  public static void main(String args[]){

      String books[]={"AI","C++","Java","Python"};

      Arrays.sort(books);

      int index=Arrays.binarySearch(books,"Python");

      System.out.println(index);
  }
}
```

Output

3

Analysis

Algorithm	Complexity
Linear Search	O(n)
Binary Search	O(log n)

Binary Search is preferred for large, sorted datasets.

Exercise 7: Financial Forecasting

Program

```
public class Forecast{

  static double futureValue(double value,int years){

     if(years==0)
        return value;

     return futureValue(value*1.10,years-1);
  }

  public static void main(String args[]){

     double result=futureValue(10000,5);

     System.out.printf("%.2f",result);
  }
}
```

Output

16105.10

(Starting amount = ₹10,000, growth rate = 10% per year for 5 years.)